

Report: .NET Versions, .NET Core, Namespace and Solution

1. .NET Versions

.NET is a software development platform and ecosystem developed by Microsoft. It provides the runtime, libraries, tools, and frameworks needed to build and run different types of applications.

.NET Framework

The first version of **.NET Framework** was released in 2002. It was mainly designed for Windows and was widely used for developing desktop and web applications.

Main characteristics:

- Windows-focused
- Mature and stable platform
- Includes the CLR and Base Class Libraries
- Mainly used today for maintaining existing or legacy applications

.NET Core

.NET Core was introduced in 2016 as a modern, cross-platform version of .NET.

Its main features are:

- **Cross-platform:** supports Windows, Linux, and macOS.
- **Open Source:** its source code is publicly available.
- **High Performance:** designed for modern and high-performance applications.
- **Modular:** applications can use the libraries and components they need.

Modern .NET

Starting with **.NET 5 in 2020**, Microsoft unified the .NET platform under the name **.NET**.

The versions then continued as:

.NET 5 → .NET 6 → .NET 7 → .NET 8 → .NET 9 → .NET 10

Modern .NET is currently the main platform for new .NET development.

LTS and STS

.NET releases have different support periods:

- **LTS (Long Term Support):** supported for a longer period, generally 3 years.
- **STS (Standard Term Support):** supported for a shorter period, generally 18 months.

For example:

.NET 8 → LTS

.NET 9 → STS

.NET 10 → LTS

2. .NET Core

.NET Core is a cross-platform and open-source development platform introduced by Microsoft in 2016.

It was developed to overcome some limitations of the traditional .NET Framework, especially its Windows-focused nature.

Main Features

1. **Cross-Platform**
Applications can run on Windows, Linux, and macOS.
 2. **Open Source**
Developers can access and contribute to its source code.
 3. **High Performance**
It was designed for modern applications and high-performance services.
 4. **Command Line Support**
Developers can create, build, run, and publish applications using the `dotnet` CLI.
-

3. Namespace

A **Namespace** is a logical way of organizing classes and other types in a C# application.

It can contain:

- Classes
- Interfaces
- Structs
- Enums
- Delegates

Why do we use Namespaces?

- Organize the code logically.
- Make large projects easier to manage.
- Prevent naming conflicts between classes with the same name.
- Allow us to access types using the `using` keyword.

Example:

```
using System;
```

Important: A namespace is a logical concept and is not necessarily the same as a physical folder.

4. Solution

A **Solution** is a container used to organize one or more related projects in a .NET application.

Example:

```
MySolution
|
├── Project1
├── Project2
└── Project3
```

A solution file can have the extension:

`.sln`

or, in newer Visual Studio versions:

`.slnx`

